

SOLO SAAS · PROJECT SPECIFICATION

BacktestLab

AI-Assisted Crypto Strategy Backtesting & Builder Platform

A no-custody, no-signals software product that lets retail crypto traders test trading strategies against real historical market data — with zero coding required.

- FastAPI + Python
- React Dashboard
- Stripe Recurring
- No Custody Risk
- ~10-Week MVP

Document Type Build Specification + AI Prompt	Target Builder Claude in VS Code	Version 1.0
---	--	-----------------------

01 Executive Summary

BacktestLab is a subscription web application that lets crypto traders answer one expensive question: *"Does this trading strategy actually work?"* Users select a trading pair, choose a pre-built strategy, tune its parameters with sliders, and run a backtest against years of historical market data. They receive an equity curve, win rate, drawdown, and risk-adjusted return — visualized in a clean dashboard.

The product was selected over crypto bots, signal groups, and tax tools because it carries **no custody risk, no financial-advice liability, and no daily content treadmill**. It is a pure software product that plays directly to a developer's strengths and generates recurring revenue after a finite, well-defined build.

~10

WEEKS TO MVP

\$29

ENTRY PRICE / MO

~\$40

MONTHLY INFRA COST

0

CUSTODY / FUNDS RISK

Positioning in one line

"The fastest way to prove — or kill — a crypto trading idea before you risk a single dollar."

02 Market & Opportunity

Factor	Assessment
Core demand	Every active trader wants to validate strategies; backtesting is a universal, evergreen need across bull and bear markets.
Buyer	Retail crypto traders, aspiring algo traders, and trading-content creators who need proof for their audiences.
Willingness to pay	Traders pay readily for tools perceived to protect or grow capital. \$29–79/mo sits in a comfortable impulse-subscription band.
Competition	Exists (TradingView strategy tester, niche tools) but most are either too complex, code-required, or not crypto-focused. The wedge is simplicity + crypto-native data .
Acquisition channels	Crypto Discords/Telegram, Reddit (r/algotrading, r/cryptocurrency), X/Twitter fin-influencers, YouTube tutorials, SEO on "[strategy] backtest".

Reality check on revenue

Headline figures like "\$10k–50k/month" are a 12-month ceiling for a well-marketed product, not a month-one outcome. A realistic early trajectory is the first paying users in months 2–3, then slow MRR growth driven by **marketing effort, not code**. Treat distribution as a permanent workstream equal in weight to the build.

03 Product Scope

3.1 MVP — In Scope (v1)

- **Historical data engine** — 2–3 years of hourly & daily candles for ~10 major pairs (BTC, ETH, SOL, BNB, XRP, etc.), stored in your own database.
- **Four pre-built strategies** — Moving-Average Crossover, RSI mean-reversion, DCA (dollar-cost averaging), and Grid.
- **Parameter tuning UI** — sliders & dropdowns (e.g. MA period, RSI thresholds, grid spacing). No user-written code.
- **Backtest results dashboard** — equity curve chart, total return %, win rate, max drawdown, Sharpe ratio, trade count, trade list.
- **Side-by-side comparison** — run two configurations and compare results (high perceived value, low build cost).
- **Auth + saved backtests** — users log in and their results persist.
- **Billing & tiers** — Stripe Checkout, free tier with monthly backtest limit, one paid tier (~\$29/mo) for unlimited.

3.2 Explicitly OUT of Scope for v1 (this is the discipline that ships)

Deferred Feature	Why it waits
Drag-and-drop strategy builder	Could double the timeline alone. Dropdowns + sliders validate the core idea first.
AI optimization suggestions	v2. Add once paying users tell you what's worth optimizing.
Multiple exchanges	One exchange (Binance) is enough to prove demand. Multi-exchange is a v2 expansion lever.
Live / paper trading	Introduces custody & liability — the exact risks this project was chosen to avoid.
User-uploaded custom code	Security rabbit hole. The "no code" angle is a feature, not a gap.

The single biggest threat to finishing

Scope creep — not picking the wrong project. Backtesting has the cleanest possible definition of "done." Guard that line ruthlessly; every deferred item above is a future revenue lever, not a launch blocker.

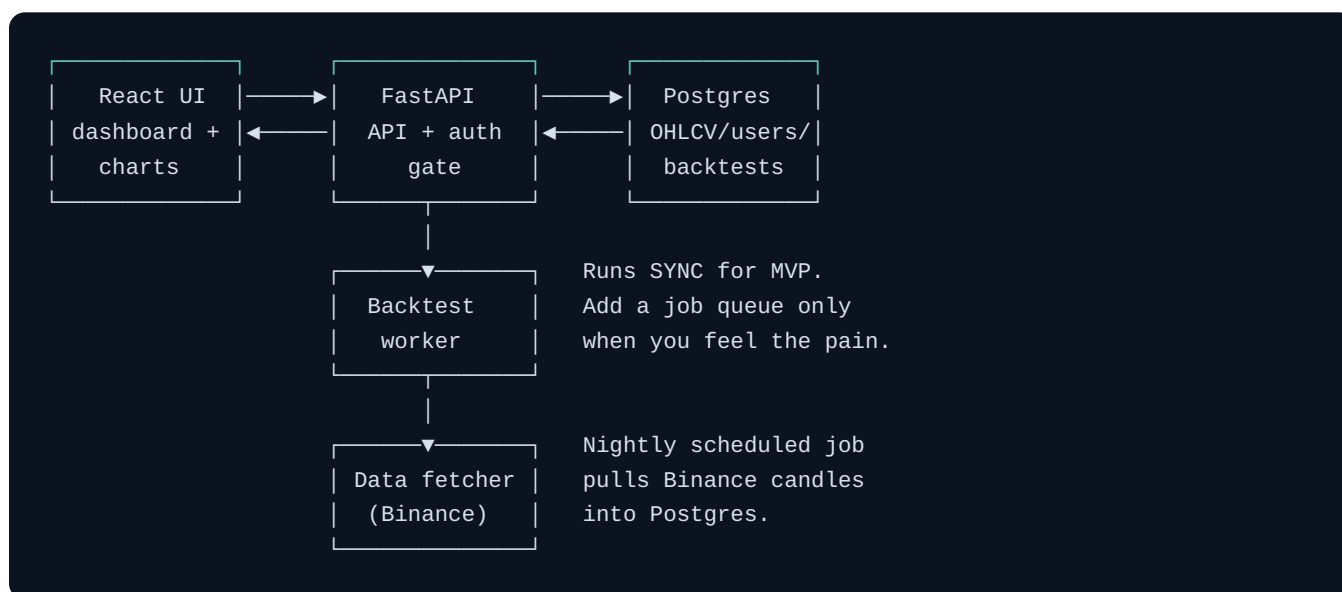
04 Technical Architecture

4.1 Stack

Layer	Choice	Rationale
Backend / API	Python + FastAPI	

		Python needed anyway for backtest math; FastAPI is fast to build and well-documented.
Backtest engine	<code>backtesting.py</code> (later <code>vectorbt</code>)	Do not build an engine from scratch. Start simple, migrate only if performance demands.
Database	PostgreSQL	Stores OHLCV candles, users, saved backtests. TimescaleDB optional later.
Market data	Binance public REST (klines)	Free, no auth for historical candles, deep history, reliable.
Frontend	React + lightweight-charts	TradingView's free charting lib; the chart is the product's face.
Auth	Clerk or Supabase Auth	Never roll your own auth.
Billing	Stripe Checkout + Customer Portal	Never build billing UI yourself.
Hosting	Railway or Render	Simpler than raw VPS for MVP (~\$40/mo). Move to a VPS later if margins require.

4.2 System Flow



4.3 Key Design Decisions

- **Pre-download data; never fetch live per backtest.** A nightly job populates Postgres. Backtests read from your DB — fast, no rate limits, no dependency on Binance during a user session.
- **Run backtests synchronously in v1.** A backtest over a few years of hourly data takes seconds. Premature job queues are scope creep.
- **Strategies are parameterized functions, not user code.** Users pick from a dropdown and tune values. Eliminates a security & complexity rabbit hole.

05 Requirements

5.1 Functional Requirements

ID	Requirement	Priority
FR-01	Fetch & store historical OHLCV for supported pairs (hourly + daily)	Must
FR-02	Run a backtest given pair, strategy, parameters, and date range	Must
FR-03	Return stats: return %, win rate, max drawdown, Sharpe, trade count, trade list	Must
FR-04	Render equity curve + stats in a dashboard	Must
FR-05	User signup / login; persist saved backtests per user	Must
FR-06	Enforce free-tier monthly backtest limit; unlock on paid plan	Must
FR-07	Stripe subscription checkout + self-serve billing portal	Must
FR-08	Side-by-side comparison of two backtest configurations	Should
FR-09	Export results (CSV/PNG)	Could

5.2 Non-Functional Requirements

Category	Target
Performance	A standard backtest (3 yrs hourly) returns in under ~5 seconds.
Reliability	Nightly data job is idempotent & retries on failure; gaps logged.
Security	Managed auth provider; no storage of exchange API keys (none needed); HTTPS everywhere; secrets in env vars.
Accuracy	Backtest must account for fees & basic slippage; clearly label assumptions to avoid misleading results.
Maintainability	Strategies are isolated modules; adding a strategy = adding one file.

5.3 Pre-Build Checklist

- ☐ Stripe account created & verified
- ☐ Binance API endpoints reviewed (klines rate limits noted)
- ☐ Auth provider (Clerk/Supabase) project created
- ☐ Hosting account (Railway/Render) + managed Postgres provisioned

- ☐ Domain name registered
- ☐ Legal disclaimer drafted ("not financial advice; past performance ≠ future results")

06 Build Plan (~10 Weeks)

Week	Focus	Deliverable
1	Data foundation	Query clean OHLCV from your own Postgres DB.
2	Backtest engine	Function: (pair, strategy, params, range) → equity curve + stats. Two strategies live (MA, RSI).
3	API layer	<code>POST /backtest</code> returns JSON. Add DCA + Grid strategies.
4	Auth + user model	Login required; results save to account.
5–6	Frontend core	Config form + results view with equity curve & stats panel; full browser flow.
7	Billing + tiers	Someone can actually pay; limits enforced.
8	Polish + landing	Loading/error/empty states, comparison view, landing page with real screenshot.
9–10	Beta + launch	10–20 beta users from crypto communities; fix & go live.

07 Risk Analysis

Likelihood & Impact rated Low / Medium / High. Score = priority of mitigation.

Risk	Likelihood	Impact	Mitigation
Scope creep (builder, AI, multi-exchange before launch)	High	High	Freeze v1 scope per §3.2. Every deferred feature is a post-launch lever. Ship the dropdown version.
Distribution failure — built but nobody finds it	High	High	Treat marketing as a permanent workstream. Start audience-building in Week 1, not after launch. Beta users become first promoters.
Misleading backtests — unrealistic results erode trust / invite complaints	Medium	High	Model fees + slippage; show assumptions; prominent "past performance ≠ future results" disclaimer.
Data pipeline fragility — Binance API changes / gaps	Medium	Medium	Idempotent retrying job; gap detection & logging; abstract the data source so a second provider can slot in.
"Passive income" illusion — underestimating support/maintenance	High	Medium	Budget ongoing time for support, churn, and API breakage. Realistic model is "build once, maintain lightly," not "fully passive."
Engine performance at scale (many users/long ranges)	Low	Medium	Sync is fine early. Migrate to <code>vectorbt</code> + a job queue only when measured latency hurts.
Competition / commoditization	Medium	Medium	Win on UX simplicity + crypto-native focus + better docs. Niche down (e.g. specific strategy families) if needed.
Regulatory drift — being seen as financial advice	Low	High	Stay a pure analytics tool; never give signals, never touch funds, never recommend trades. Clear ToS + disclaimers.
Solo-founder burnout / stalled finish	Medium	High	Weekly deliverables (\$6) create momentum & visible progress. Ship beta at Week 9 even if imperfect.

Top three to internalize

- 1) Protect the v1 scope line.
 - 2) Start marketing on day one.
 - 3) Make backtests honest (fees + disclaimers).
- These three decide success more than any code choice.

08 Monetization

Tier	Price	Includes
------	-------	----------

Free	\$0	Limited backtests/month, core strategies, single-config only — the funnel.
Pro	~\$29/mo	Unlimited backtests, all strategies, side-by-side comparison, saved history, export.
Elite (v2)	~\$79/mo	AI optimization suggestions, multi-exchange data, priority features — introduced once validated.

Annual plans (2 months free) improve cash flow and reduce churn once the product proves sticky.

09 Ready-to-Use Prompt for Claude in VS Code

Paste the block below into Claude inside VS Code to kick off the build. It encodes the stack, scope guardrails, and the first concrete task so the agent doesn't over-build.

```
# PROJECT: BacktestLab – Crypto Strategy Backtesting SaaS
```

```
You are my senior engineer building an MVP. Follow scope strictly – do NOT add features beyond what I list. Ask before introducing any new dependency.
```

```
## PRODUCT
```

```
A web app where retail crypto traders pick a trading pair, choose a pre-built strategy, tune parameters with sliders, and backtest it against historical Binance data. They get an equity curve + stats. No custody, no live trading, no signals, no user-written code.
```

```
## STACK (do not deviate without asking)
```

- Backend: Python + FastAPI
- Backtest engine: backtesting.py (DO NOT build an engine from scratch)
- DB: PostgreSQL
- Market data: Binance public REST klines (no auth needed for historical)
- Frontend: React + lightweight-charts (TradingView)
- Auth: Clerk OR Supabase (managed – never roll our own)
- Billing: Stripe Checkout + Customer Portal
- Hosting target: Railway/Render

```
## V1 SCOPE – IN
```

1. Data fetcher: pull 2-3 yrs hourly+daily candles for ~10 major pairs into Postgres
2. Four strategies: MA crossover, RSI mean-reversion, DCA, Grid (parameterized functions)
3. Backtest engine wrapper -> returns: return %, win rate, max drawdown, Sharpe, trade count, trade list, equity curve series. MUST include fees + basic slippage.
4. FastAPI endpoint POST /backtest (sync is fine)
5. Auth + per-user saved backtests
6. React dashboard: config form (pair, strategy, params, date range) + results view
7. Side-by-side comparison of two configs
8. Stripe: free tier (monthly backtest limit) + Pro (~\$29/mo unlimited)

```
## V1 SCOPE – OUT (do NOT build these)
```

- Drag-and-drop strategy builder
- AI optimization
- Multiple exchanges
- Live/paper trading
- User-uploaded code
- Job queues / async workers (run sync until I say otherwise)

```
## START HERE (Task 1 of the plan)
```

```
Set up the repo structure, FastAPI skeleton, and PostgreSQL schema for: candles(pair, interval, open_time, o, h, l, c, v), users, and saved_backtests. Then write the Binance data fetcher that backfills hourly + daily candles for BTCUSDT and ETHUSDT into Postgres, with: idempotent upserts, gap detection, and retry on failure. Give me the file tree first, then the code, then exact run instructions. Confirm the schema with me before writing the fetcher.
```

RULES

- Prioritize a working end-to-end slice over completeness.
- Flag any assumption explicitly.
- Keep each strategy in its own module so adding one = adding one file.
- Always include a "not financial advice / past performance != future" disclaimer in any results output.

How to drive the build

Work one weekly deliverable at a time (§6). After each task lands and runs, tell Claude "proceed to the next task" and reference the week. Resist the urge to pull v2 features forward — that discipline is what gets this shipped and earning.